43

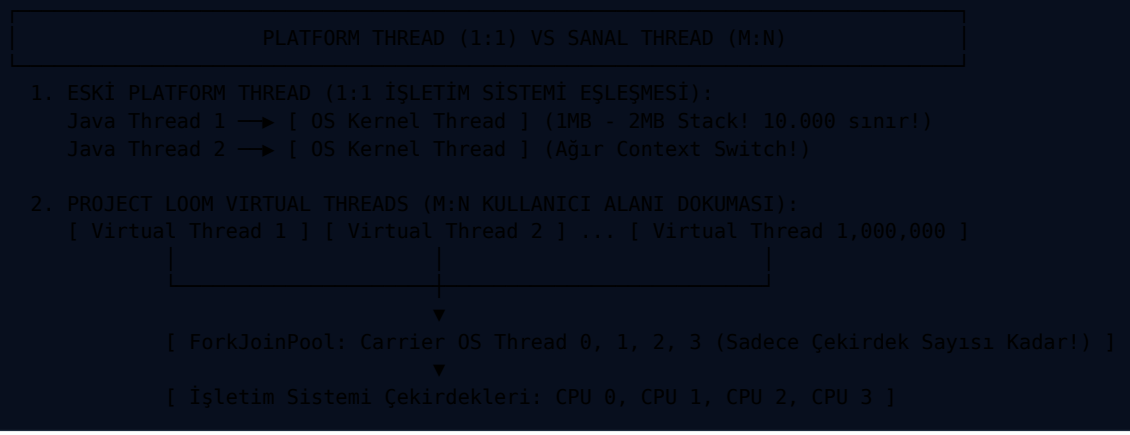
BÖLÜM: PROJECT LOOM & SANAL THREAD'LER (VIRTUAL THREADS) | MODULE: PROJECT LOOM & VIRTUAL THREADS

Platform Thread vs Sanal Thread: M:N Mimarisi & Milyonlarca Eşzamanlı İş Parçacığı

Platform Threads vs Virtual Threads: M:N Fiber Architecture

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Platform thread, her posta paketi için şirkete 10 tonluk devasa bir TIR kiralamak gibidir. Garaja ancak 5000 TIR sığar; her TIR 2 MB yakıt yakar ve şoför değiştirmek dakikalar sürer. Virtual Thread (Sanal Thread) ise aynı TIR'ın içine konan milyonlarca hafif mektup zarfıdır! Mektup okunurken veya cevap beklenirken TIR durmaz; postacı hemen diğer zarfı açar. 1 milyon zarfı cebinde taşıyabilirsin, bilgisayarının nefesi bile kesilmez!

🇬🇧 ENGLISH (ELI5)

A Platform Thread is like chartering an entire 18-wheel cargo truck for every single envelope you mail. You run out of parking spots after a few thousand trucks, each consumes 1MB-2MB of stack space, and switching drivers is an expensive OS kernel interrupt. Virtual Threads are the individual letters inside a handful of cargo trucks. If a letter is waiting for a signature, the truck driver simply sets it aside and reads the next letter. You can spawn 1,000,000 virtual threads without breaking a sweat!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

HotSpot M:N Fiber Tasarımı: 1) Bellek Verimliliği: Klasik platform thread'leri OS düzeyinde rezerve edilen 1 MB - 2 MB sabit yığın (stack) alanı tüketir. 10.000 thread 10-20 GB RAM demektir. Sanal thread'ler ise JVM heap alanında yaşar; ilk oluşturklarında yalnızca birkaç yüz baytlık bir Java nesnesidir (`java.lang.VirtualThread`). Yığınları çağrı derinliğine göre dinamik olarak büyür ve küçülür. 2) M:N Eşleme: Milyonlarca sanal thread (M), CPU çekirdek sayısı kadar platform thread'i (`Carrier Thread` - N) üzerinde ForkJoinPool zamanlayıcısıyla çalıştırılır. 3) Thread-per-Request Paradigması: Asenkron callback cehennemine (reactive spaghetti) girmeden, tamamen senkron ve okunabilir kod yazarak reaktif mimarilerin devasa verimine ulaşılır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

HotSpot M:N Fiber Architecture: 1) Memory Footprint: Platform threads bind 1:1 to OS kernel threads, reserving 1MB to 2MB of fixed contiguous off-heap stack memory. Spawning 10,000 platform threads demands 15 GB RAM and triggers OS thread limits (`OutOfMemoryError: unable to create native thread`). Virtual threads allocate dynamic stack frames directly on the JVM heap (`Continuation`), starting at under 1 KB. 2) M:N Scheduling: Millions of virtual threads (M) are multiplexed over a pool of N kernel carrier threads (typically matching CPU core count) managed by an internal FIFO `ForkJoinPool`. 3) Synchronous Simplicity with Reactive Scale: Engineers retain the familiar thread-per-request paradigm with synchronous stack traces and debuggers while achieving 100K+ concurrent I/O throughput.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: 100.000 Sanal Thread Başlatma ve Hız Karşılaştırması
import java.time.Duration;
import java.time.Instant;
import java.util.concurrent.Executors;
import java.util.stream.IntStream;

public class VirtualThreadScaleDemo {
    void main() {
        System.out.println("Java 25 Sanal Thread Testi Başlatılıyor...");
        Instant start = Instant.now();

        // 100.000 eşzamanlı I/O işini Sanal Thread'lerle yürüt
        try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
            IntStream.range(0, 100_000).forEach(i -> {
                executor.submit(() -> {
                    // Bloklayan I/O simülasyonu (Ağ/DB gecikmesi)
                    Thread.sleep(Duration.ofMillis(100));
                    return i;
                });
            });
        } // Executor close() tüm 100.000 thread'in bitmesini bekler!

        Instant end = Instant.now();
        System.out.printf("100.000 sanal thread %.2f saniyede başarıyla tamamlandı!%n",
            Duration.between(start, end).toMillis() / 1000.0);
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Sanal thread'leri ASLA havuzlamayın (pool)! newFixedThreadPool(100) gibi sanal thread havuzları açmak ölümcül bir anti-pattendir. Sanal thread'ler o kadar ucuzdur ki her iş için yenisi yaratılmalı ve iş bitince ölmelerine izin verilmelidir (Disposable Task Runners). Sınırlandırma gerekiyorsa havuz değil Semaphore kullanılmalıdır!

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

NEVER pool Virtual Threads! Creating a fixed virtual thread pool is a catastrophic anti-pattern. Virtual threads are designed to be ephemeral and disposable: instantiate one per task, let it terminate, and allow GC collection. To constrain concurrency or access to scarce downstream resources (e.g. database connections), use a `Semaphore`, never a thread pool!

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- Platform thread'leri 1:1 OS thread'idir ve 1 MB yer kaplar. Sanal thread'ler JVM heap'inde yaşar, yüzlerce bayttır ve milyonlarcası açılabilir.

🎯 Key Takeaways for the Interview

- Platform threads bind 1:1 to costly OS kernel threads. Virtual threads are lightweight heap-allocated fibers multiplexed over carrier threads.